# HUST

**ĐẠI HỌC BÁCH KHOA HÀ NỘI**

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

# MAXIMUM A POSTERIOR


## Group 13:
Đặng Anh Tú
Chu Đức Nam
Bùi Thanh Thạch
Trương Việt Hoàng
Nguyễn Gia Vinh


ONE LOVE. ONE FUTURE.

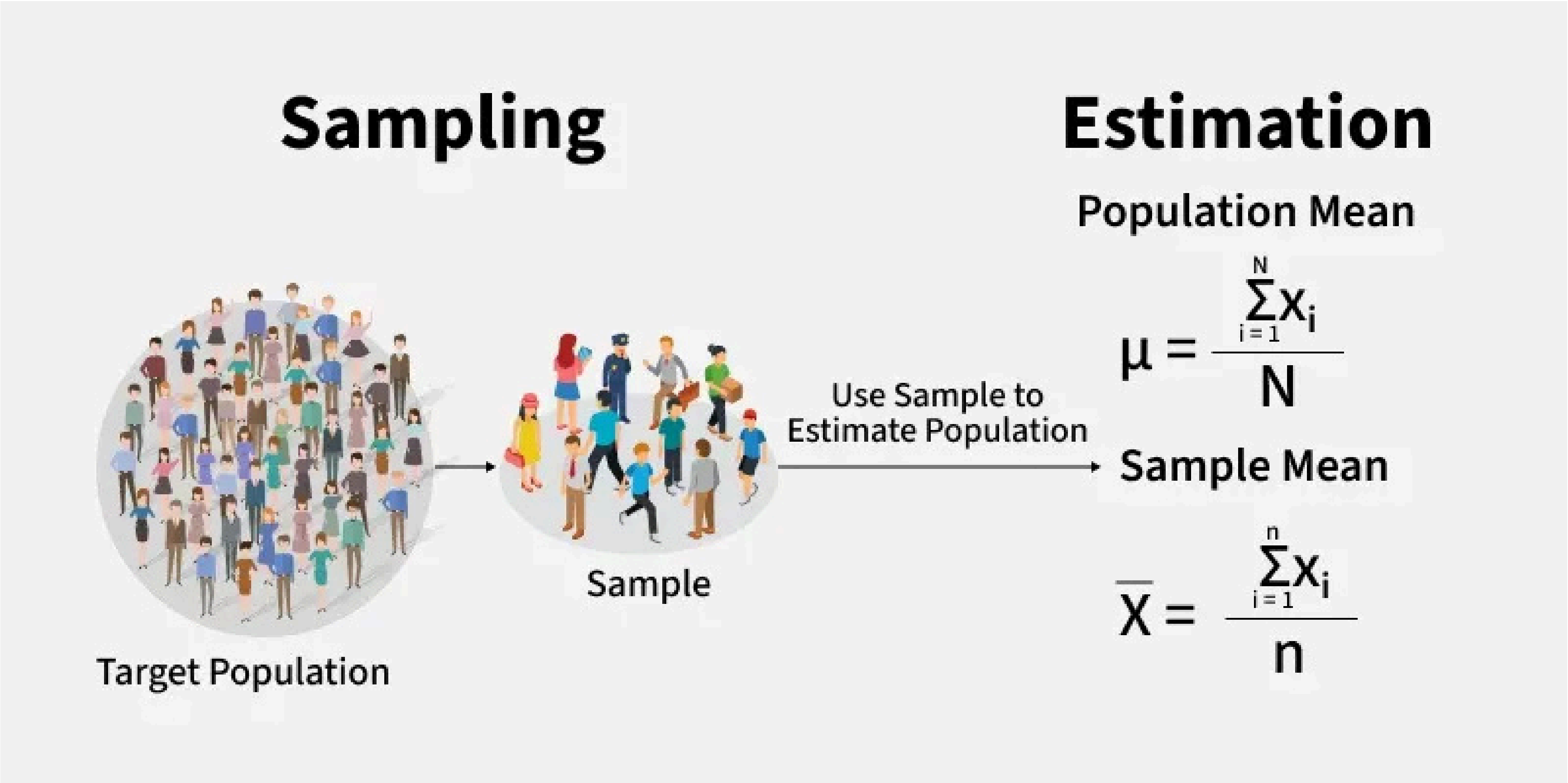
ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

# Introduction

## Statistical Estimation & MAP Framework

ONE LOVE. ONE FUTURE.

# What is Statistical Estimation?




- **Goal:**

Given observed data, find the parameter θ that best explains it.

- **Core question:**

*"Which value of θ fits the data most?"*

- **Two main schools of thought:**
  - **Frequentist →** Maximum Likelihood Estimation (MLE)
  - **Bayesian →** Maximum A Posteriori (MAP), Full Bayesian

# MAP Estimation

- **Bayes' Theorem:**

$$\underbrace{p(\theta|\boldsymbol{y})}_{\text{Posterior}} = \frac{\underbrace{p(\theta)}_{\text{Prior}}\underbrace{p(\boldsymbol{y}|\theta)}_{\text{Likelihood}}}{\underbrace{p(\boldsymbol{y})}_{\text{Marginal likelihood}}}$$

- **MAP objective:** Find θ that maximizes the posterio

$$\theta_{MAP} = \arg\max_{\theta} p(\theta|y) = \arg\max_{\theta} \log p(\theta|y) = \arg\max_{\theta} \log \frac{p(\theta)p(\boldsymbol{y}|\theta)}{p(\boldsymbol{y})}$$

- **Intuition:** Balance between *evidence* from *data* and *prior belief* about θ

- **Why log?** Convert products → sums
  - Avoid numerical underflow
  - Easier to differentiate

# Project Concept

**GOAL**

Estimate the Maximum A Posteriori (MAP) value of insurance claim using machine learning approaches that combine prior knowledge and observed data.

## PROBLEM OVERVIEW

Given historical insurance data and prior information, the project estimates the most probable claim amount (MAP) using different modeling approaches.

## KEY APPROACHES

**Coin Flipping (Baseline)**
Simple probabilistic baseline using prior likelihood.

**Sensor Fusion**
Combine multiple data sources to improve estimation.

**Linear Regression**
Predict claim value using linear regression model.

## WORKFLOW

**Input Data**
Historical claims + prior knowledge (sensor data, etc.)

→

**Models**
- Coin Flipping
- Sensor Fusion
- Linear Regression

→

**MAP Estimation**
Compute MAP estimate for claim amount

→

**Evaluation**
Evaluate and compare model performance

## EXPECTED OUTCOMES

- Improve estimation accuracy
- Compare performance of different approaches
- Provide insights for insurance decision-making

# Implementation Details

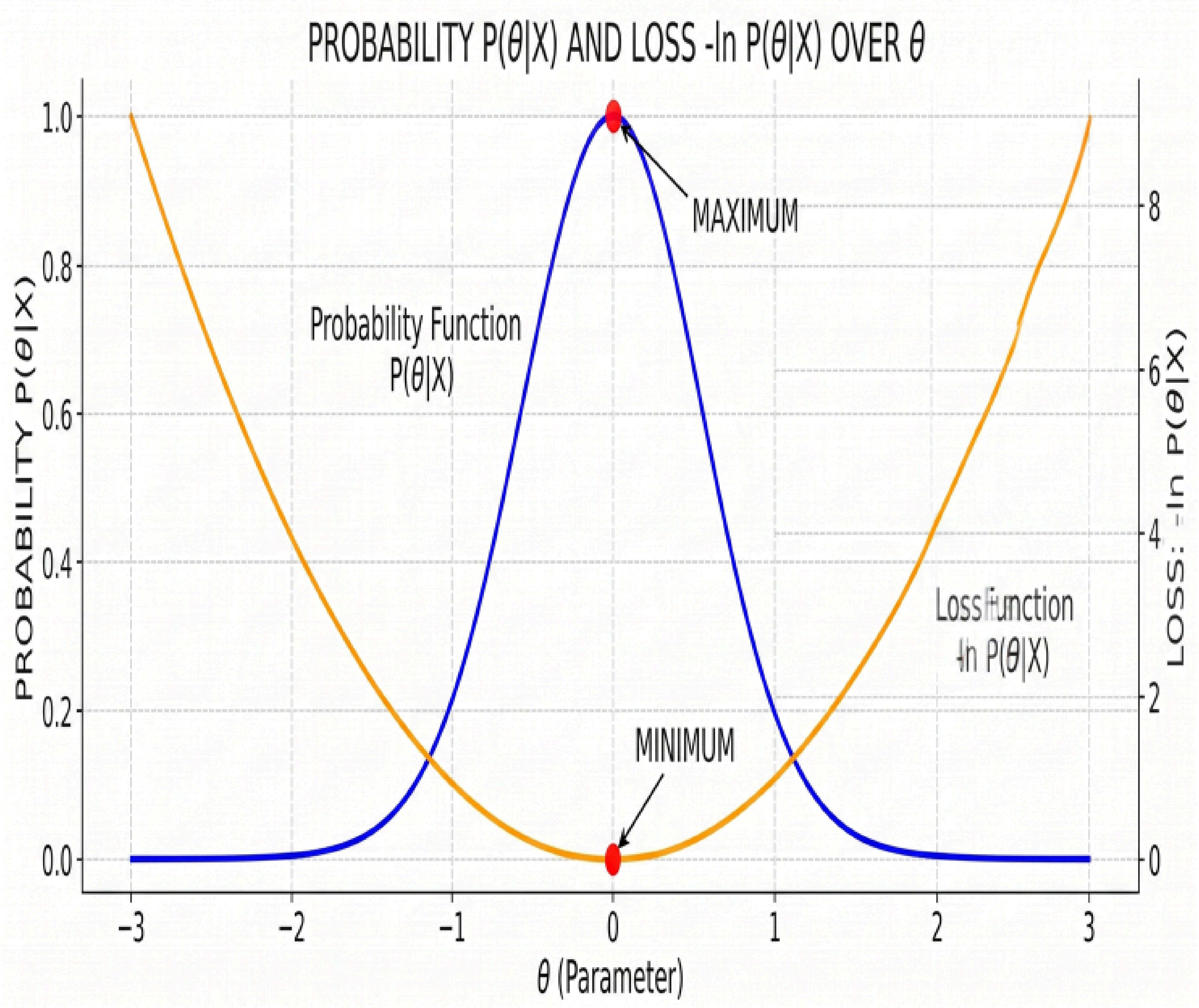
# Mathematical Transformation - Why use Gradient Descent?

- The objective of MAP (Maximum A Posteriori): Find the parameter θ that maximizes the posterior probability $P(\theta|X)$

- Problem transformation: From Maximize Probability →Minimize Negative Log-Posterior (Loss Function).

- Decomposition formula:

$$\text{Loss}(\theta) = -\log P(X|\theta) - \log P(\theta)$$



# Building the Optimizer (GradientDescentOptimizer)

- **Hyperparameters:** learning_rate, ($\alpha$)epochs, tol (convergence threshold).
- **Update Rule:**

$$\theta_{new} = \theta_{old} - \alpha \cdot \nabla_{\theta \text{Loss}}(\theta)$$

- **Convergence Check:** $||\theta_{new} - \theta_{old}|| < \text{tol}$

```python
import numpy as np

class GradientDescentOptimizer:
    def __init__(self, learning_rate=0.01, epochs=1000, tol=1e-6):
        self.lr = learning_rate
        self.epochs = epochs
        self.tol = tol

    def minimize(self, theta_init, grad_func, data, **kwargs):
        """
        General Gradient Descent Algorithm.
        Goal: Find theta to minimize the loss function (Negative Log-Posterior).
        """
        theta = np.array(theta_init, dtype=float)

        for i in range(self.epochs):
            # Compute the gradient at the current point
            grad = grad_func(theta, data, **kwargs)

            # Update parameters (stepping in the opposite direction of the gradient)
            new_theta = theta - self.lr * grad

            # Check for convergence
            if np.linalg.norm(new_theta - theta) < self.tol:
                break
            theta = new_theta

        return theta
```

# Calculating Gradients for 3 Examples (Gradients Implementation)

- **Example 1 (Coin Flipping):** Bernoulli + Beta. Technical note: np.clip the parameter θ within the range $(0,1)$ to avoid division-by-zero errors.

- **Example 2 (Sensor Fusion):** Gaussian Gaussian. The derivative is the linear sum of the Likelihood residual and the Prior residual.

- **Example 3 (Ridge Regression):** Linear + Gaussian.

$$\nabla_w \text{Loss} = \frac{X^T(Xw - y)}{\sigma^2} + \lambda w$$

```python
# Clip theta within (0, 1) to avoid log errors or division by zero
theta = np.clip(theta, 1e-10, 1 - 1e-10)
```

```python
    # Derivative of the -Log Likelihood (Gaussian)
    grad_likeli = np.sum(theta - data) / (sigma_likeli**2)
    # Derivative of the -Log Prior (Gaussian)
    grad_prior = (theta - mu_prior) / (sigma_prior**2)

    return grad_likeli + grad_prior
```

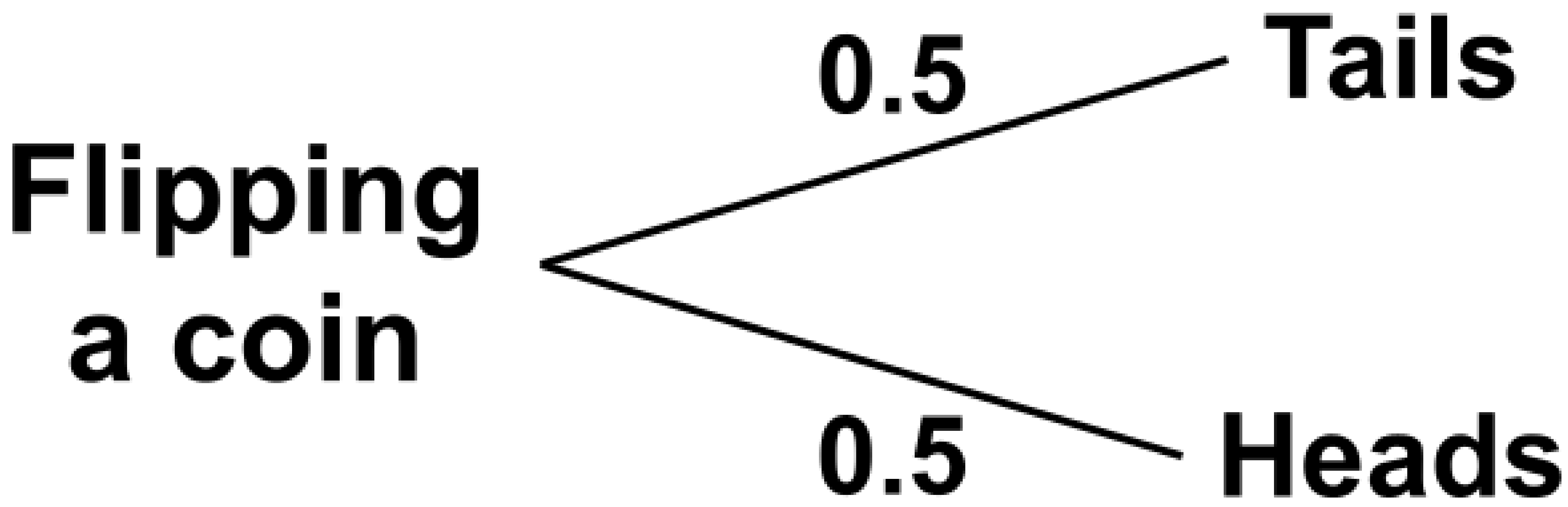
```python
# Add a column of ones to account for the bias term (intercept w0)
X_b = np.column_stack([np.ones(len(X)), X])

# Ridge derivative formula: X.T @ (Xw - y) / sigma^2 + lambda * w
prediction_error = X_b @ w - y
gradient = (X_b.T @ prediction_error) / (sigma**2) + lmbda * w
return gradient
```

# Example 1

## Compare MAP and MLE through the coin flipping problem

# Coin Flipping Problem




- We flip a coin **n** times
- Each result is represented as: 1 = Head; 0 = Tail
- The goal is to estimate: **θ** = probability of getting Head

# Method Ideas

1. **Maximum Likelihood Estimation (MLE)**

- MLE estimates the parameter using only the observed data.
- For the Bernoulli coin flipping problem:

$$\theta_{MLE} = \frac{k}{n}$$

**with k = Number of heads**
**n = Observations**

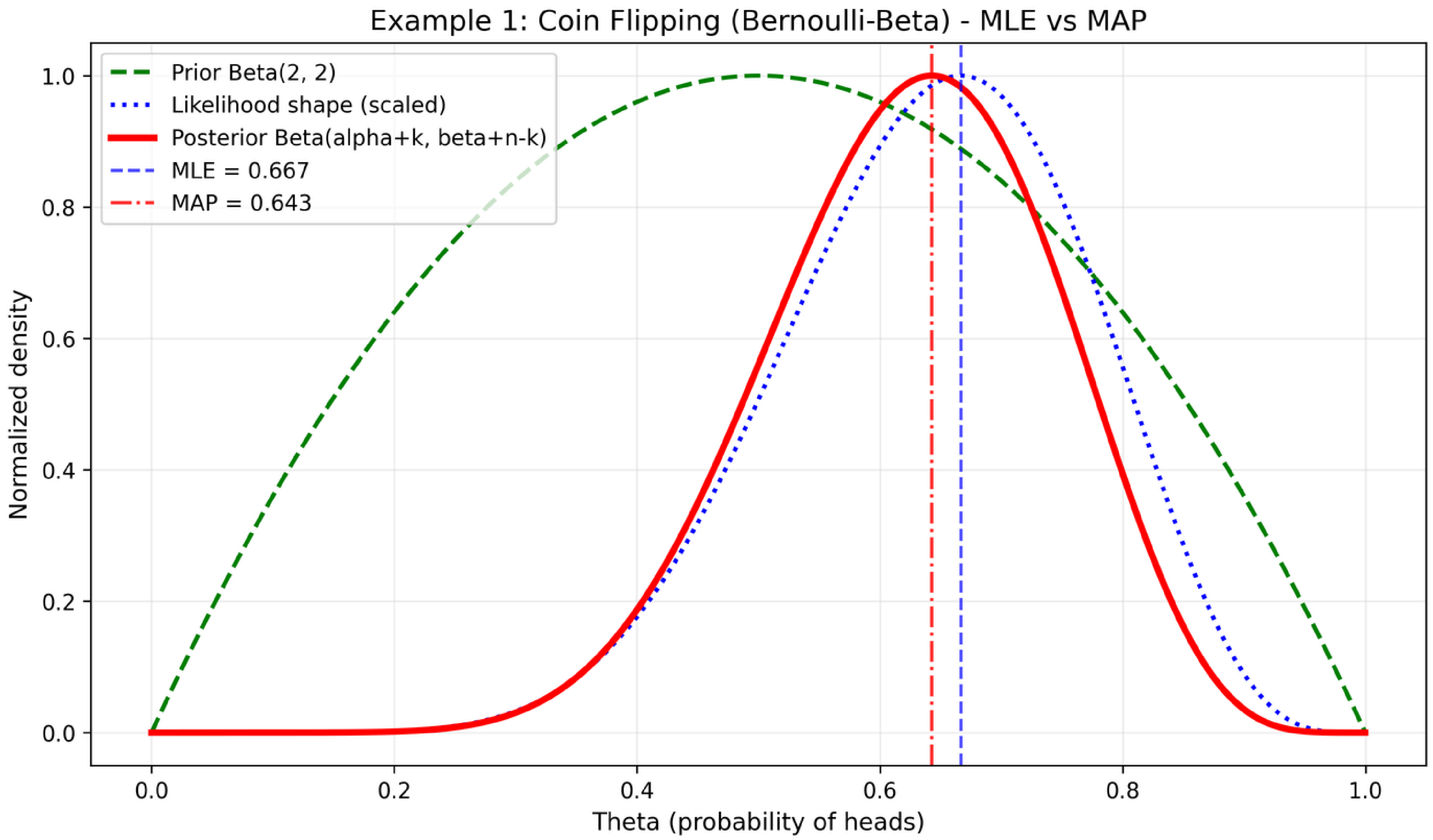
2. **Maximum A Posteriori Estimation (MAP)**

- MAP combines: **Observed data + Prior belief**
- We use a Beta prior: $\theta \sim Beta(\alpha, \beta)$
- The MAP formula:

$$\theta_{MAP} = \frac{k + \alpha - 1}{n + \alpha + \beta - 2}$$

# Visual Comparison

- Observations (n): 12
- Heads (k): 8
- Tails (n-k): 4
- Beta prior: α = 2, β = 2




- The MLE estimate is closer to the observed data.
- The MAP estimate is influenced by both the data and the prior belief.
- Because the prior Beta(2,2) favors values near 0.5, MAP is slightly pulled toward the center.

# Pros/ Cons between MLE and MAP

| | MLE | MAP |
|---|---|---|
| **Pros** | • Very simple; often closed-form and fast.<br>• Uses only observed data, easy to interpret. | • Combines data evidence with prior belief.<br>• More robust in low-data settings (regularization effect). |
| **Pros** | • Can be unstable with small samples.<br>• No mechanism to include prior domain knowledge. | • Requires selecting a prior (can introduce bias if poorly chosen).<br>• May require numerical optimization in complex models. |

## Conclusion:

- **MLE is suitable when we have a large amount of data.**
- **MAP is useful when the dataset is small or when prior knowledge is available.**

# Example 2

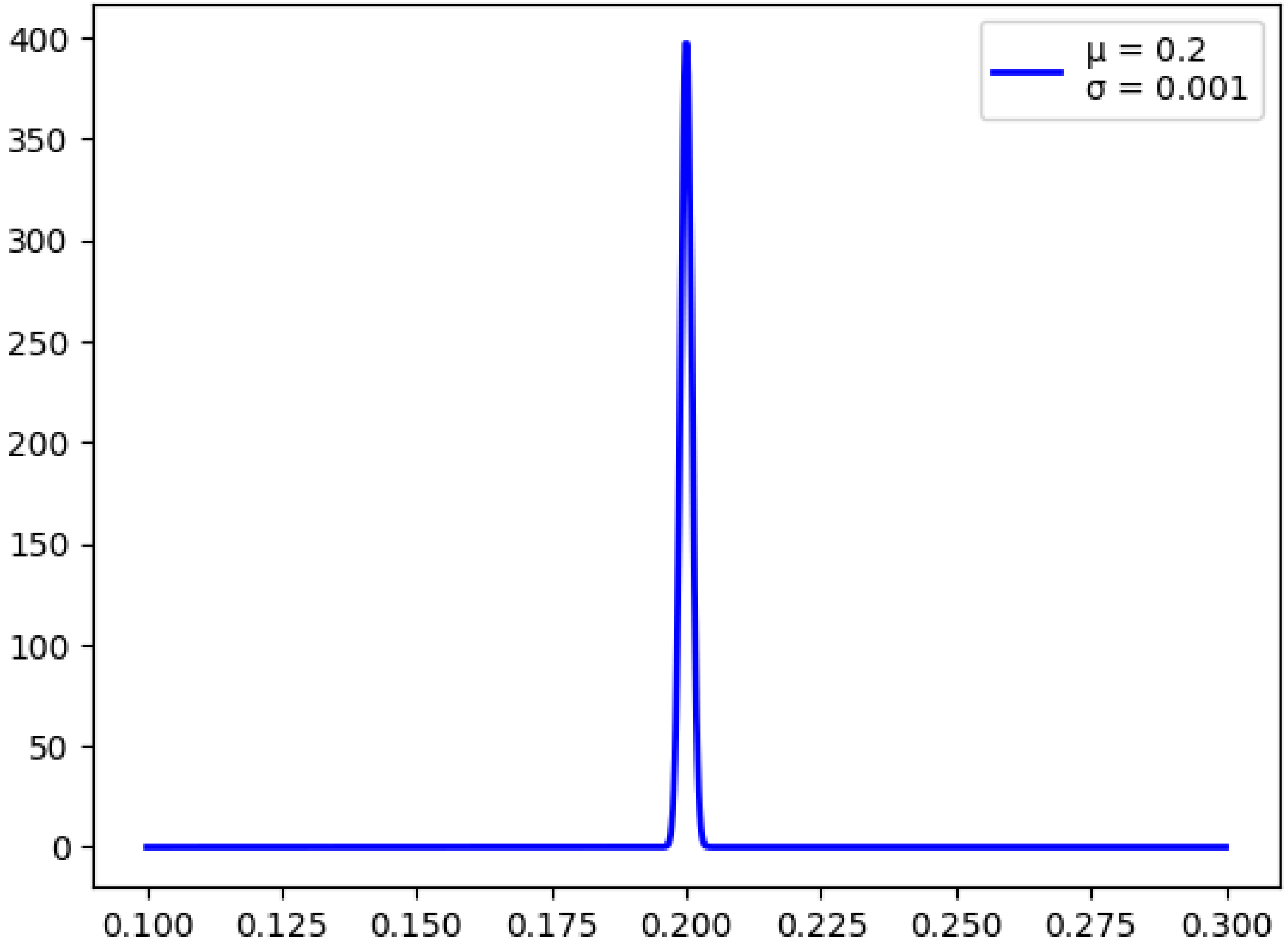
## High-Precision Sensor Calibration in Noisy Environments

# Problem Statement - Systemic Bias Estimation

| Temperature | Sensor1 | Sensor2 | Sensor3 | Sensor4 | Sensor5 |
|---|---|---|---|---|---|
| 25,43149 | 0,677916 | 0,208399 | 0,033098 | 0,145758 | 0,291933 |
| 24,48606 | 0,801851 | 0,445989 | 0,869082 | 0,499015 | 0,53149 |
| 20,05509 | 0,781817 | 0,040596 | 0,205773 | 0,517225 | 0,598457 |
| 22,78169 | 0,65103 | 0,316018 | 0,945797 | 0,433242 | 0,378673 |
| 25,29283 | 0,292357 | 0,893463 | 0,289188 | 0,490347 | 0,012502 |

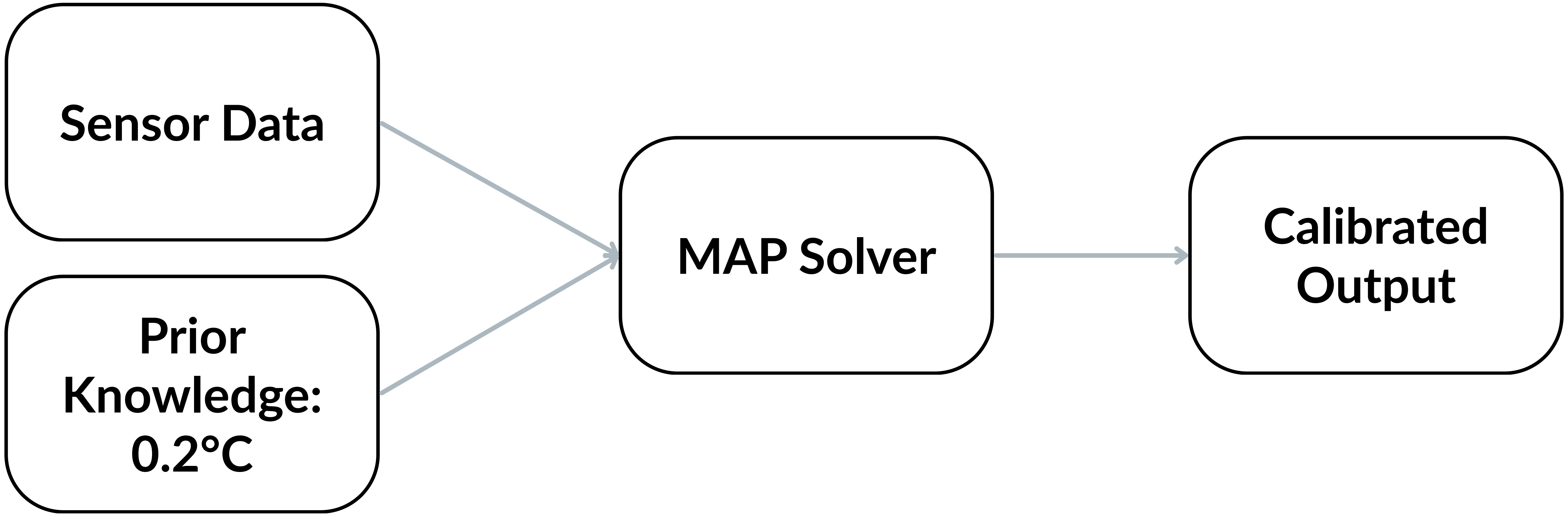
- Objective: Identify fixed systemic bias for precise sensor calibration.
- Dataset: 17,000 records from 5 independent sensors.
- Challenge: Stochastic noise makes simple MLE (averaging) unreliable.

# Problem Statement - Systemic Bias Estimation



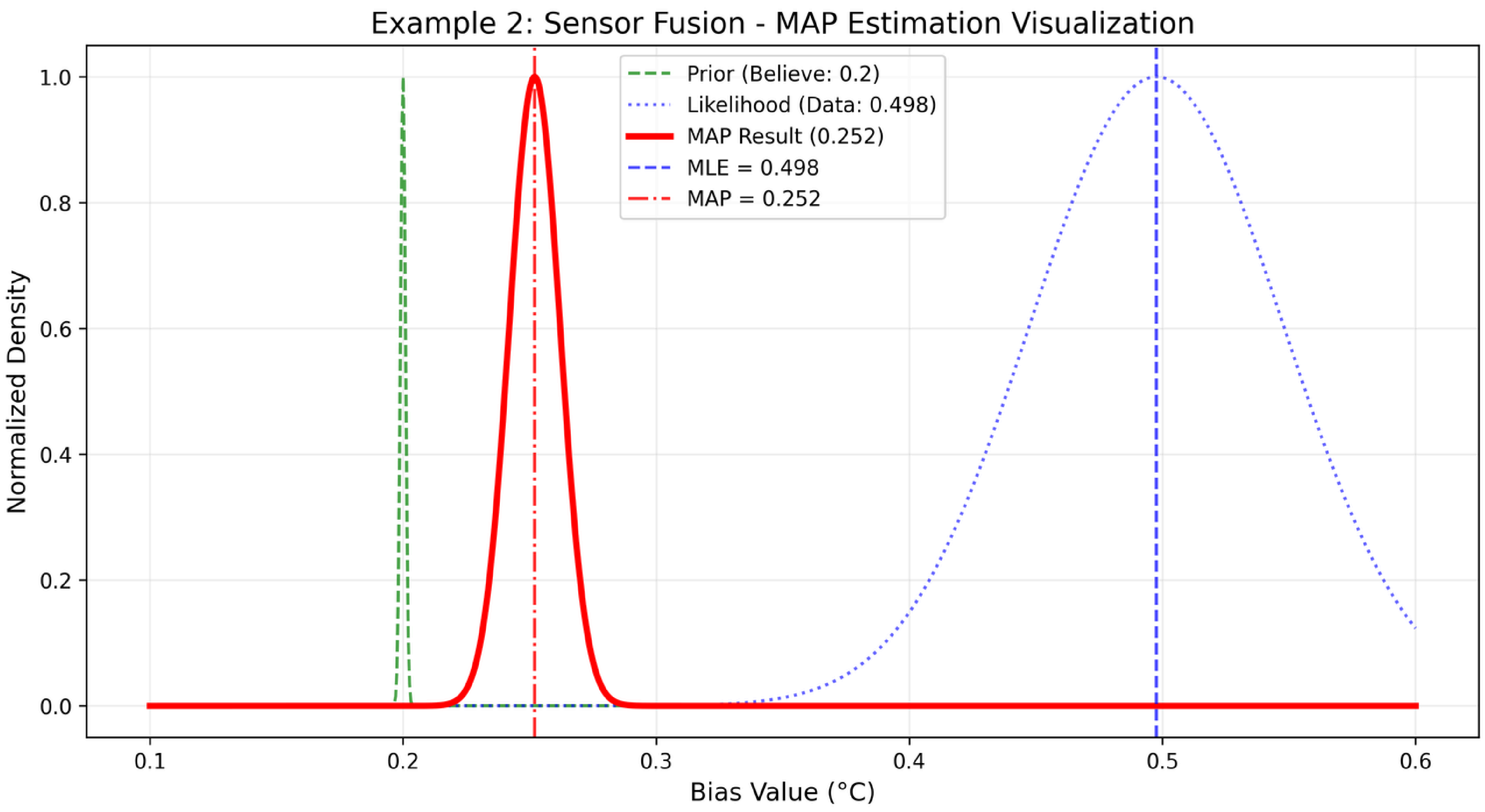

- Prior Assumption ($\mu_{prior}$): Fixed at +0.2°C, representing high-end sensor technology.
- Confidence Level ($\sigma_{prior}$): Set at 0.001, reflecting absolute trust in technical specs.
- Significance: Acts as a mathematical anchor to resist transient measurement noise.

# MAP Framework & Implementation




- Probabilistic Model: Log-Gaussian Likelihood for sensor noise modeling.
- Optimization Strategy: Custom Gradient Descent for objective minimization.
- System Integration: Modular pipeline for automated data processing.

# Visual Analysis: MLE vs. MAP Results




- Likelihood (Blue): Peaks at 0.498°C — Severe MLE bias due to noisy data.
- MAP (Red): Peaks at 0.252°C — Optimal balance via expert prior.
- Core Takeaway: MAP filters environmental noise to stabilize the system near the 0.2°C baseline.

# Example 3

## Predicting parameters in linear regression

# Problem Statement - Weights Estimation

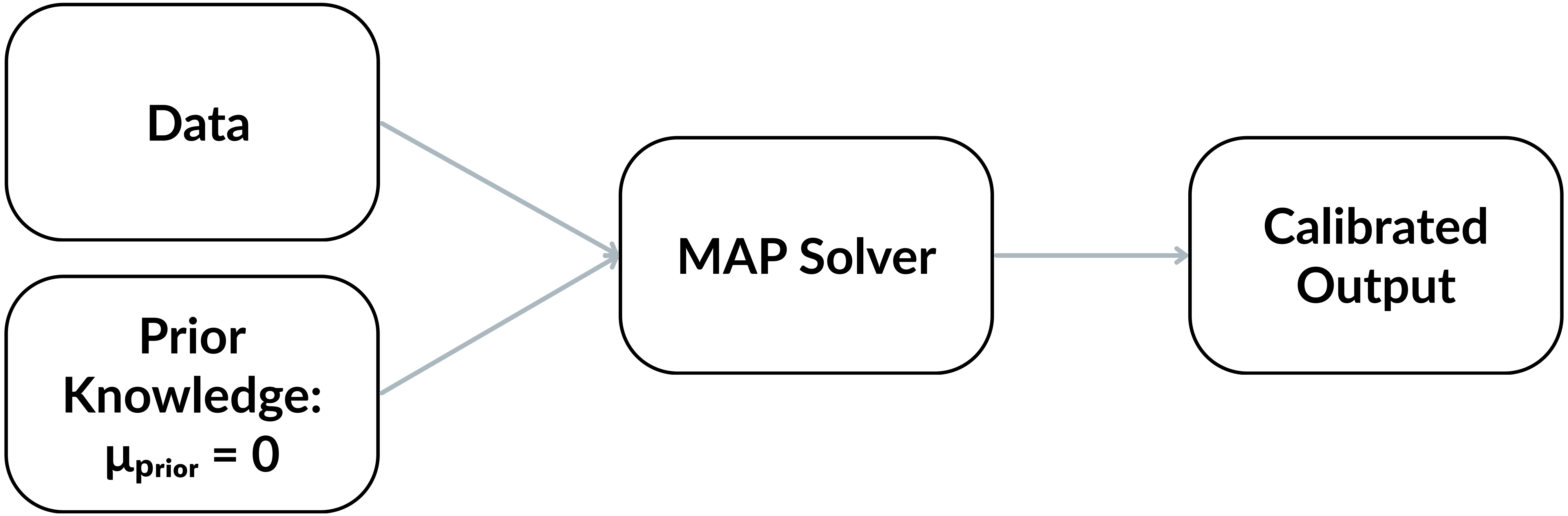
| 1 | age | sex | bmi | children | smoker | charges | region_northwest | region_southeast | region_southwest | charges_log |
|---|---|---|---|---|---|---|---|---|---|---|
| 2 | -1.440417729 | 0 | -0.453159594 | -0.90923416 | 1 | 16884.924 | FALSE | FALSE | TRUE | 9.734176431773115 |
| 3 | -1.51164747 | 1 | 0.50942164544 | -0.07944162 | 0 | 1725.5523 | FALSE | TRUE | FALSE | 7.453302452133774 |
| 4 | -0.799350061 | 1 | 0.38315460035 | 1.5801434662 | 0 | 4449.462 | FALSE | TRUE | FALSE | 8.400538468975023 |
| 5 | -0.443201357 | 1 | -1.305052191 | -0.90923416 | 0 | 21984.471 | TRUE | FALSE | FALSE | 9.998091601725266 |
| 6 | -0.514431098 | 1 | -0.292456082 | -0.90923416 | 0 | 3866.8552 | TRUE | FALSE | FALSE | 8.260196845858372 |
| 7 | -0.585660839 | 0 | -0.807363253 | -0.90923416 | 0 | 3756.6216 | FALSE | TRUE | FALSE | 8.231275322 |
| 8 | 0.48278527438 | 0 | 0.45530719754 | -0.07944162 | 0 | 8240.5896 | FALSE | TRUE | FALSE | 9.016827173741753 |
| 9 | -0.158282394 | 0 | -0.479396902 | 1.5801434662 | 0 | 7281.5056 | TRUE | FALSE | FALSE | 8.893092932994646 |
| 10 | -0.158282394 | 1 | -0.136672066 | 0.7503509232 | 0 | 6406.4107 | FALSE | FALSE | FALSE | 8.765054439884247 |
| 11 | 1.48000164671 | 0 | -0.790964936 | -0.90923416 | 0 | 28923.137 | TRUE | FALSE | FALSE | 10.272397139270046 |
| 12 | -1.013039284 | 1 | -0.728651329 | -0.90923416 | 0 | 2721.3208 | FALSE | FALSE | FALSE | 7.908872629665523 |
| 13 | 1.62246112847 | 0 | -0.717172507 | -0.90923416 | 1 | 27808.725 | FALSE | TRUE | FALSE | 10.233105102955316 |
| 14 | -1.155498766 | 1 | 0.61273104596 | -0.90923416 | 0 | 1826.843 | FALSE | FALSE | TRUE | 7.510344619461671 |
| 15 | 1.19508268319 | 0 | 1.50151985686 | -0.90923416 | 0 | 11090.718 | FALSE | TRUE | FALSE | 9.313863803 |

- Objective: Predicting parameters for linear regression analysis of the problem.
- Dataset: Information of 1337 households (standardized).
- Challenge: Stochastic noise makes simple MLE (averaging) unreliable.

# Problem Statement - Weights Estimation

- Prior Assumption ($\mu_{prior}$): Set at 0, assuming the features have very little impact on the result.
- Confidence Level ($\sigma_{prior}$): Set at 0.4, reflecting belief in a priori knowledge.
- Significance: acts as a neutral prior belief that features should not strongly influence the target unless supported by data, while simultaneously serving as a mathematical anchor to resist transient measurement noise by constraining coefficients toward zero.

# MAP Framework & Implementation

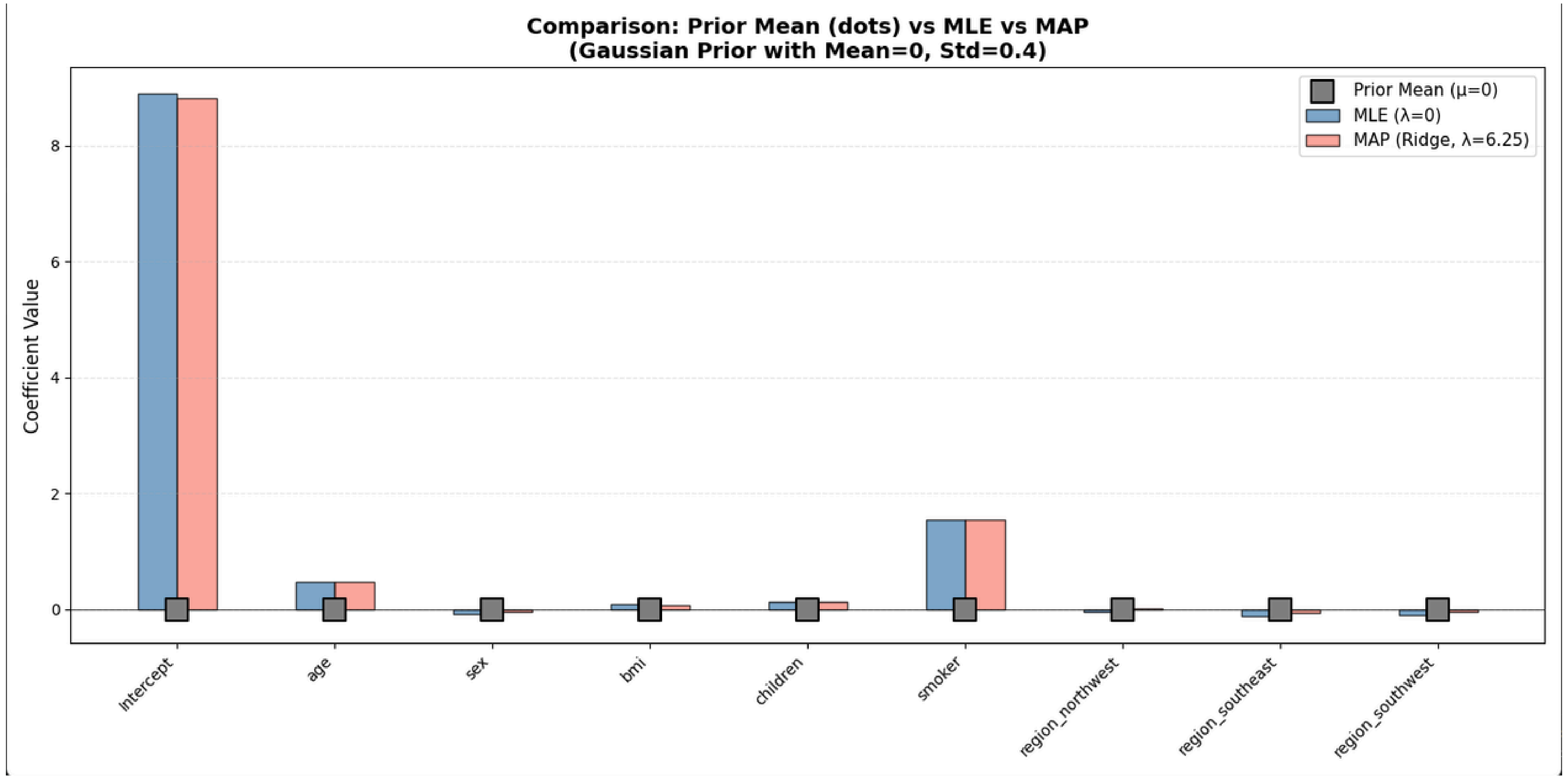


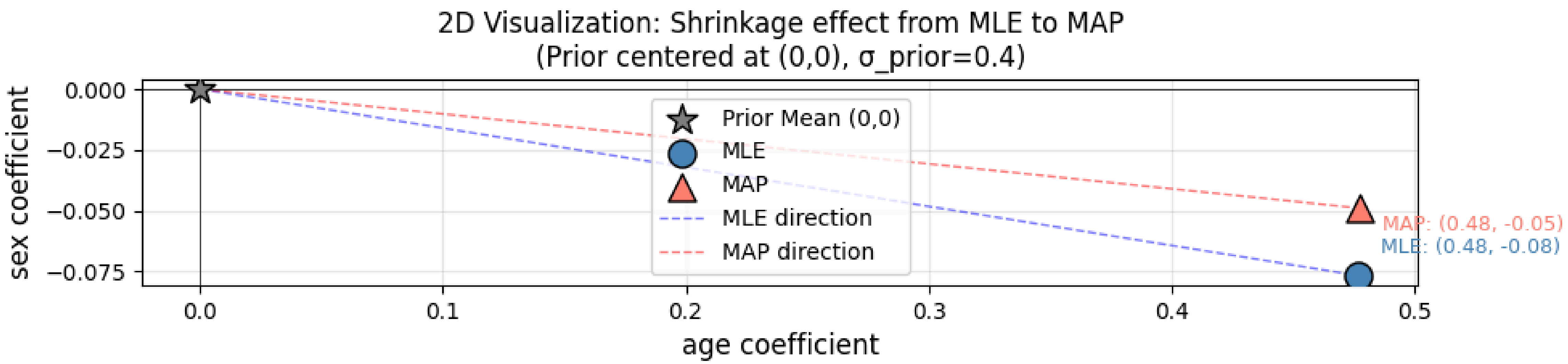

- Probabilistic Model: Gaussian Prior and MAP-based Log-Linear Regression for robust insurance cost modeling under stochastic noise.
- Optimization Strategy: Custom Gradient Descent for MAP objective minimization with Ridge-style regularization.
- Bayesian Integration: Combines observed data with prior knowledge ($\mu_{prior} = 0$) to produce noise-resistant calibrated predictions.

# Visual Analysis: MLE vs. MAP Results

- Compared to MLE, MAP is somewhat more 'hesitant' because it has a priori information that features do not affect the results.

- Sometimes this can help avoid overfitting.



# Conclusion

The project has successfully implemented the MAP estimation framework, proving it to be a powerful tool due to its ability to seamlessly integrate empirical data (Likelihood) with prior knowledge (Prior). Through three experimental problems, we draw the following core conclusions:

- Effective with small & noisy data: MAP (using Beta/Gaussian Priors) corrects the extreme estimates of MLE in small-sample coin flipping and filters out environmental noise in sensor calibration.
- Preventing Overfitting: In Linear Regression, MAP is equivalent to Ridge regression (L2 Regularization), which controls weight magnitudes and enhances the model's generalization capabilities.
- Stable Optimization: Utilizing the Negative Log-Posterior objective function combined with Gradient Descent provides a continuous, accurate, and efficient parameter optimization process.

**Takeaway Message:** In data science, integrating domain expertise (Prior) through MAP mathematics is the key to building robust and trustworthy AI models.